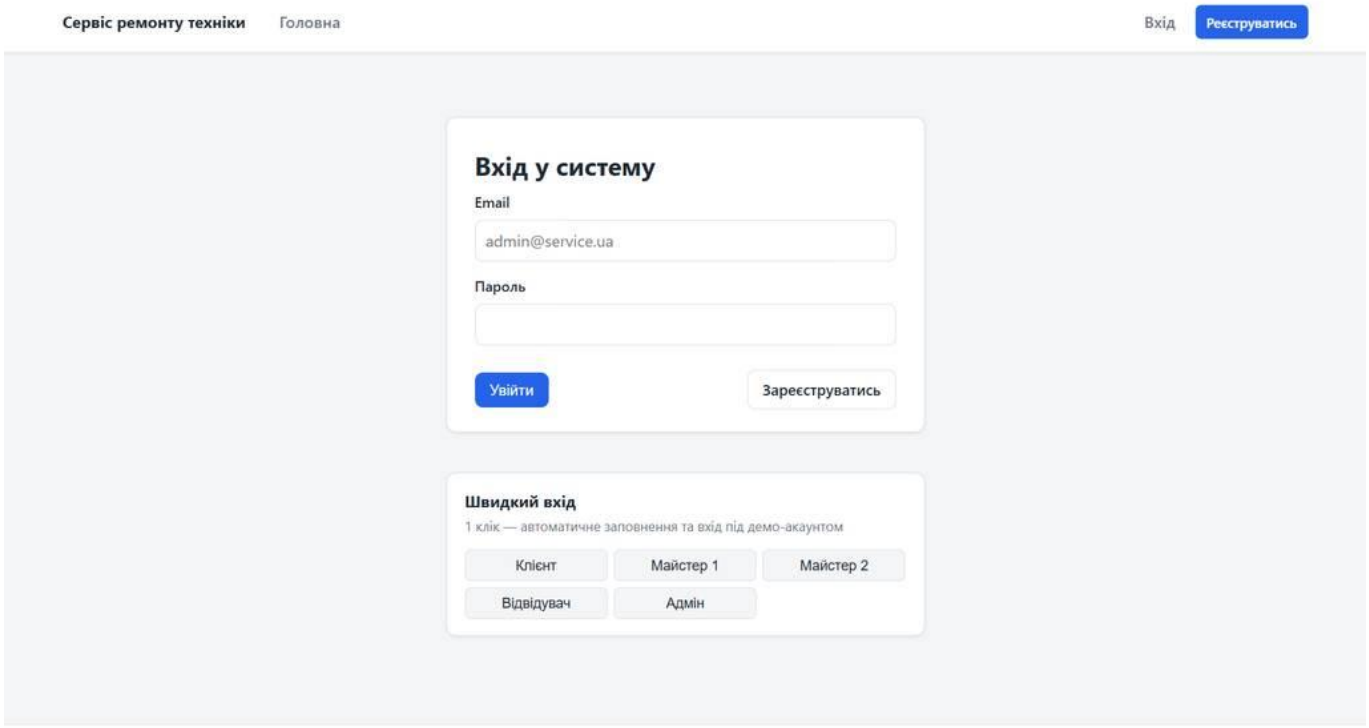
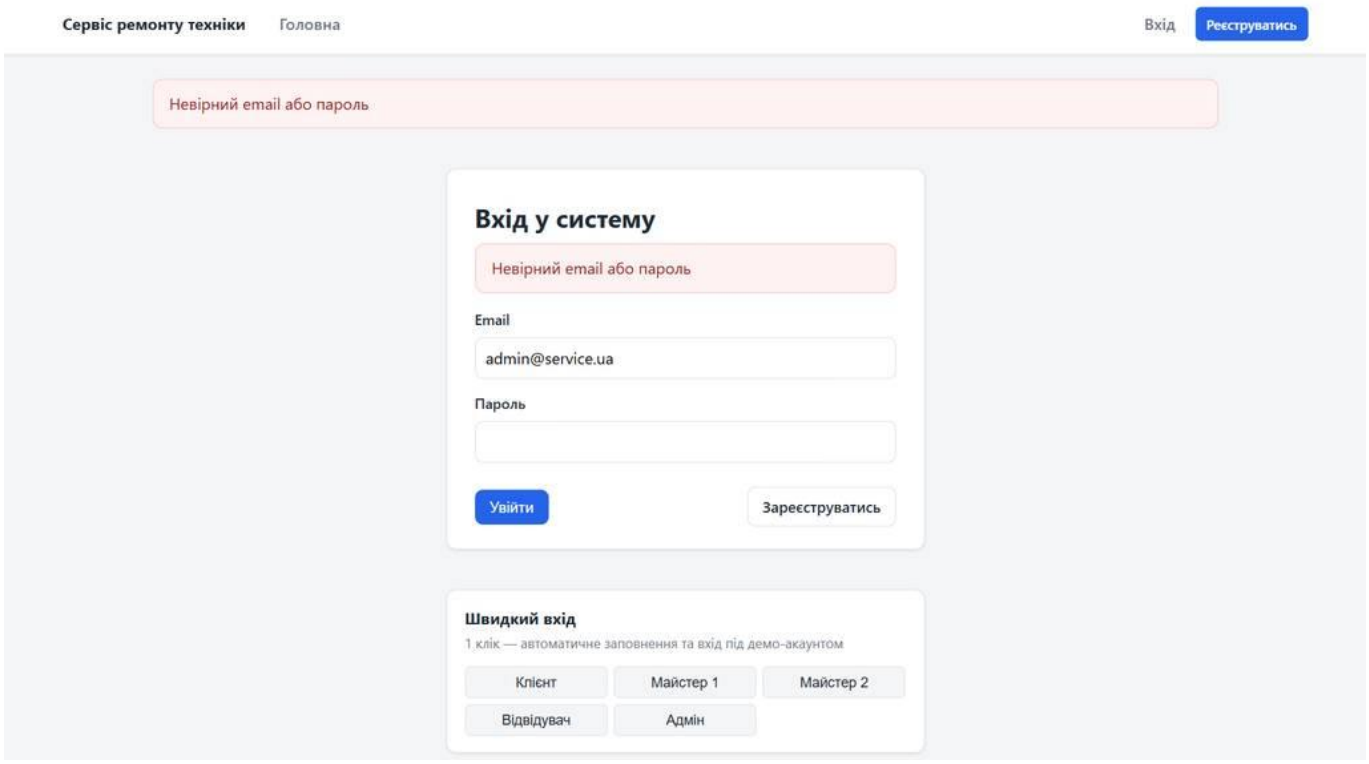
ЗВІТ
про виконання лабораторної роботи №4
«Авторизація та контроль доступу»
з дисципліни
«Вебтехнології»
Студента групи КН-2327Б
Багрія-Белза Андрія

Мета роботи: Реалізувати авторизацію користувачів (Passport.js, express-session, bcryptjs) та контроль доступу за ролями: четверта роль Відвідувач, захист мутуючих маршрутів, сторінка 403.

Завдання 1: Вхід та вихід користувачів: форма /login (email + пароль), локальна стратегія Passport.js, перевірка хеша bcryptjs, сесії express-session; публічна реєстрація з присвоєнням ролі Відвідувач.



Сервіс ремонту та обслуговування комп'ютерної техніки · Lab 2



Форма входу та відмова за невірними даними: статус 401 і повторний рендер форми з банером «Невірний email або пароль». У сесії зберігається лише id користувача (serializeUser/deserializeUser), а роль читається з БД при кожному запиті, тому зміна ролі в адмінці діє негайно. Cookie сесії: httpOnly, sameSite=lax, живучість 8 годин.

Завдання 2: Захист маршрутів за ролями: middleware requireRole(roles) — неавторизованого редіректить на /login?next=..., авторизованого без потрібної ролі — на сторінку 403.

1. src/lib/auth.ts — middleware захисту маршруту

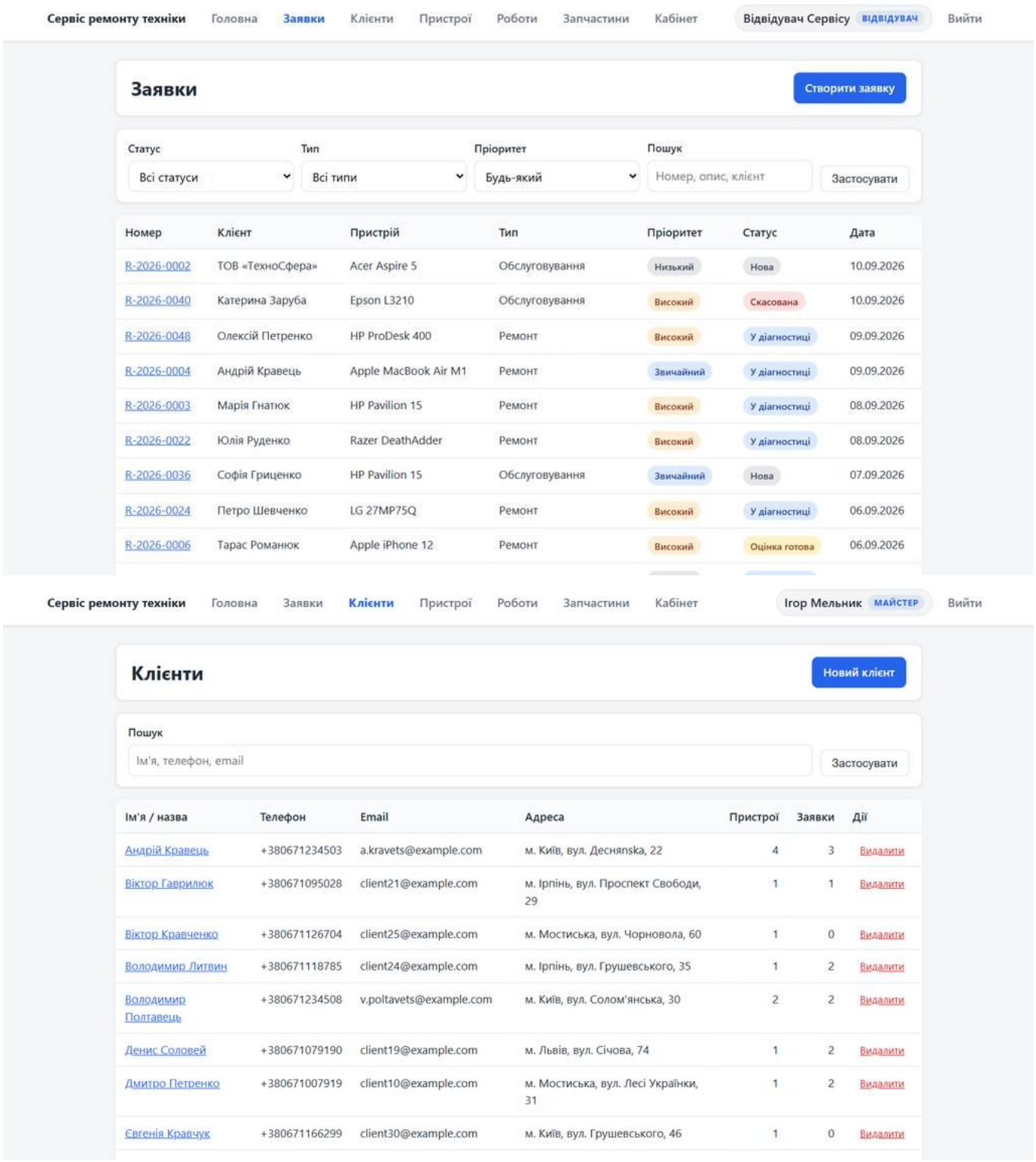
```
export function requireRole(roles: readonly string[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user) {
      const nextUrl = req.originalUrl;
      return res.redirect(`/login?next=${encodeURIComponent(nextUrl)}`);
    }
    if (!roles.includes(req.user.role)) {
      return res
        .status(403)
        .render("errors/403", {
          nav: "none",
          requiredRoles: roles.map((r) => ROLE_LABELS[r] ?? r).join(", "),
        });
    }
    next();
  };
}
```

2. src/routes/clients.ts — матриця дозволів на маршрутах

```
import { Router } from "express";
import { index, show, createForm, create, editForm, edit, remove, confirmRemove } from
"../controllers/clients.js";
import { requireRole, ROLES } from "../lib/auth.js";

// Матриця дозволів (Lab 4): перегляд — персонал + відвідувач; зміни — персонал
(адмін/майстер).
const router = Router();
router.get("/", requireRole(ROLES.VIEW), index);
router.get("/new", requireRole(ROLES.STAFF), createForm);
router.post("/", requireRole(ROLES.STAFF), create);
router.get("/:id", requireRole(ROLES.VIEW), show);
router.get("/:id/edit", requireRole(ROLES.STAFF), editForm);
router.post("/:id/edit", requireRole(ROLES.STAFF), edit);
```

```
router.post("/:id/delete", requireRole(ROLES.STAFF), remove);
router.get("/:id/delete", requireRole(ROLES.STAFF), confirmRemove);
```



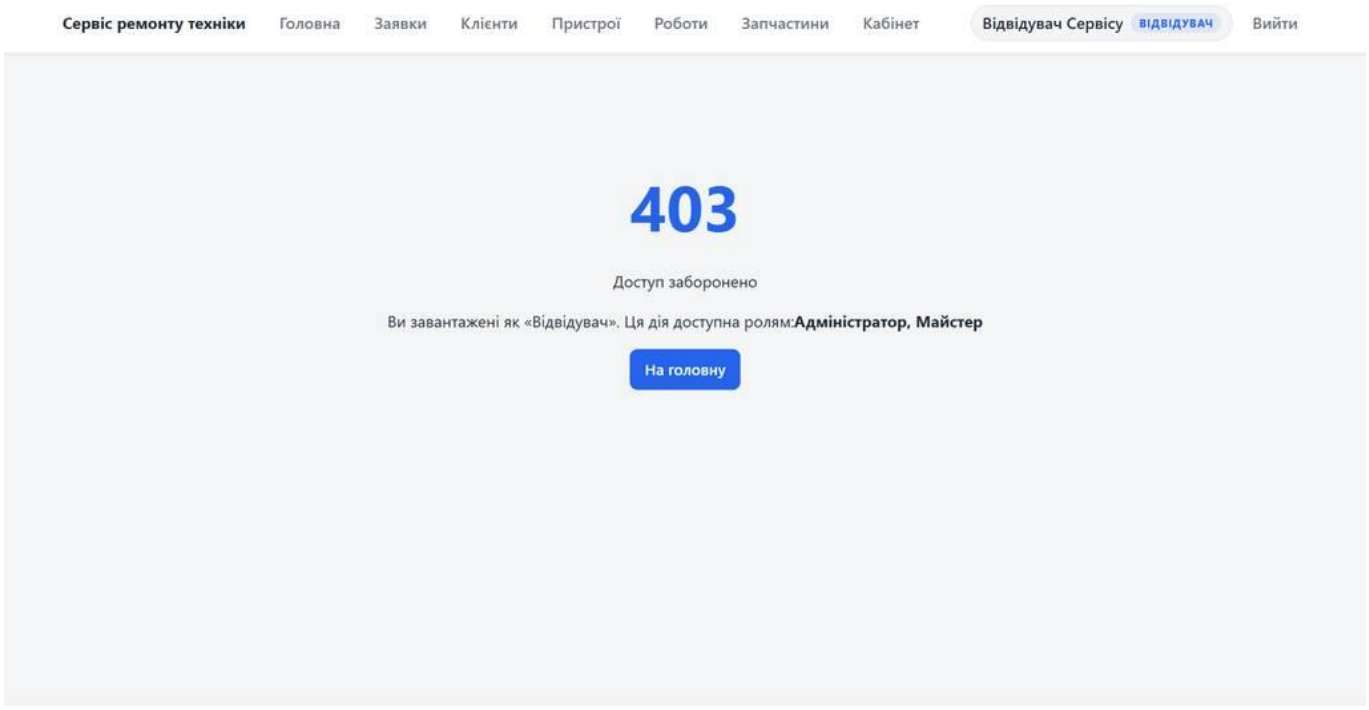
Відвідувач (роль VISITOR) має лише перегляд: список заявок відкривається без входу та після входу, а спроба перейти на створення заявки завершується сторінкою 403, де названо, яка роль потрібна. Гість натомість отримує редірект на /login?next=... і після входу повертається на цю саму сторінку.

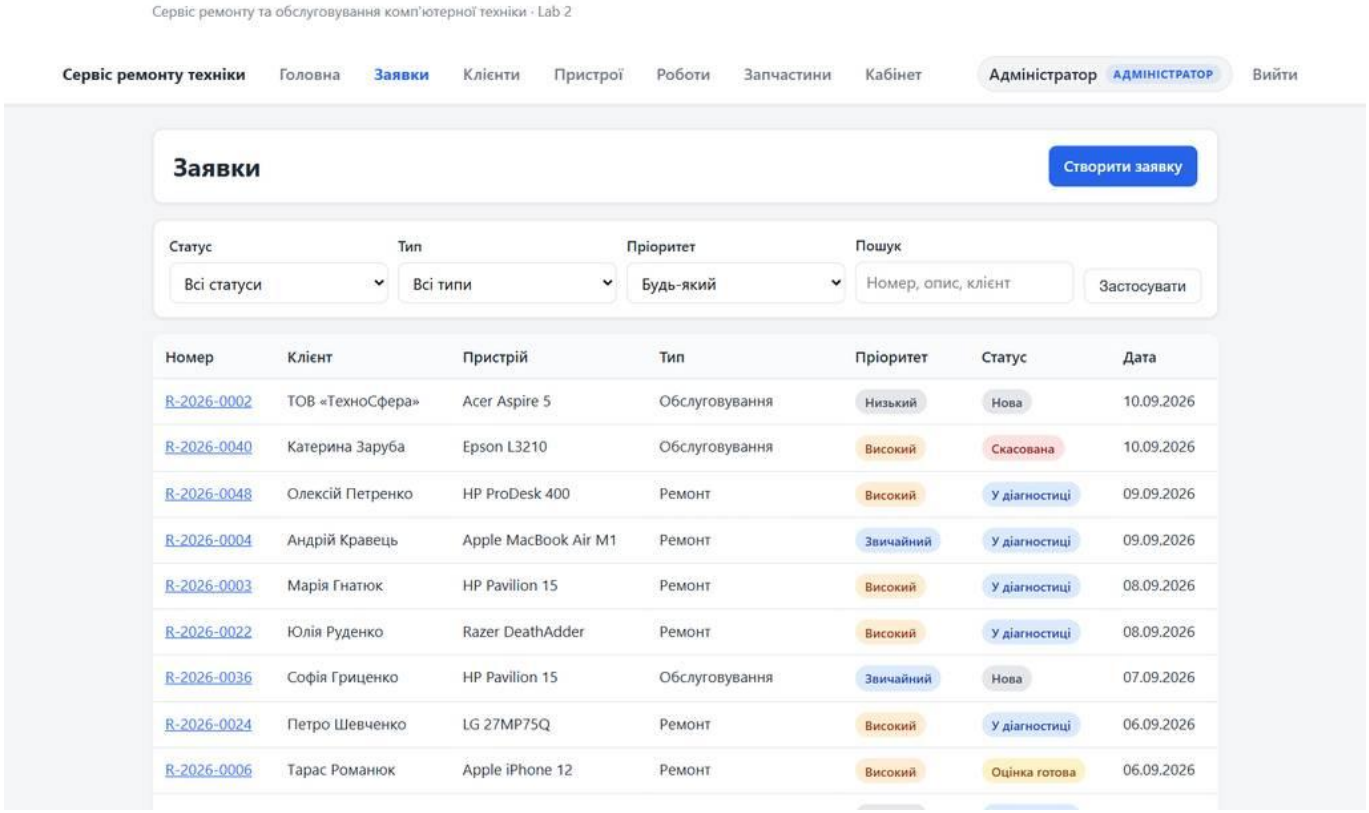
Завдання 3: Матриця дозволів для 4 ролей і role-aware інтерфейс: меню й дії відображаються лише тим, хто може їх виконати; фінальний захист — завжди на сервері.

1. src/lib/auth.ts — групи ролей

```
export const ROLES = {
```

```
  /** Персонал: адмін + майстер (каталоги, заявки, фінанси, технічні дії). */
  STAFF: ["ADMIN", "TECHNICIAN"],
  /** Персонал + відвідувач (усі сторінки даних у режимі перегляду). */
  VIEW: ["ADMIN", "TECHNICIAN", "VISITOR"],
} as const;
```



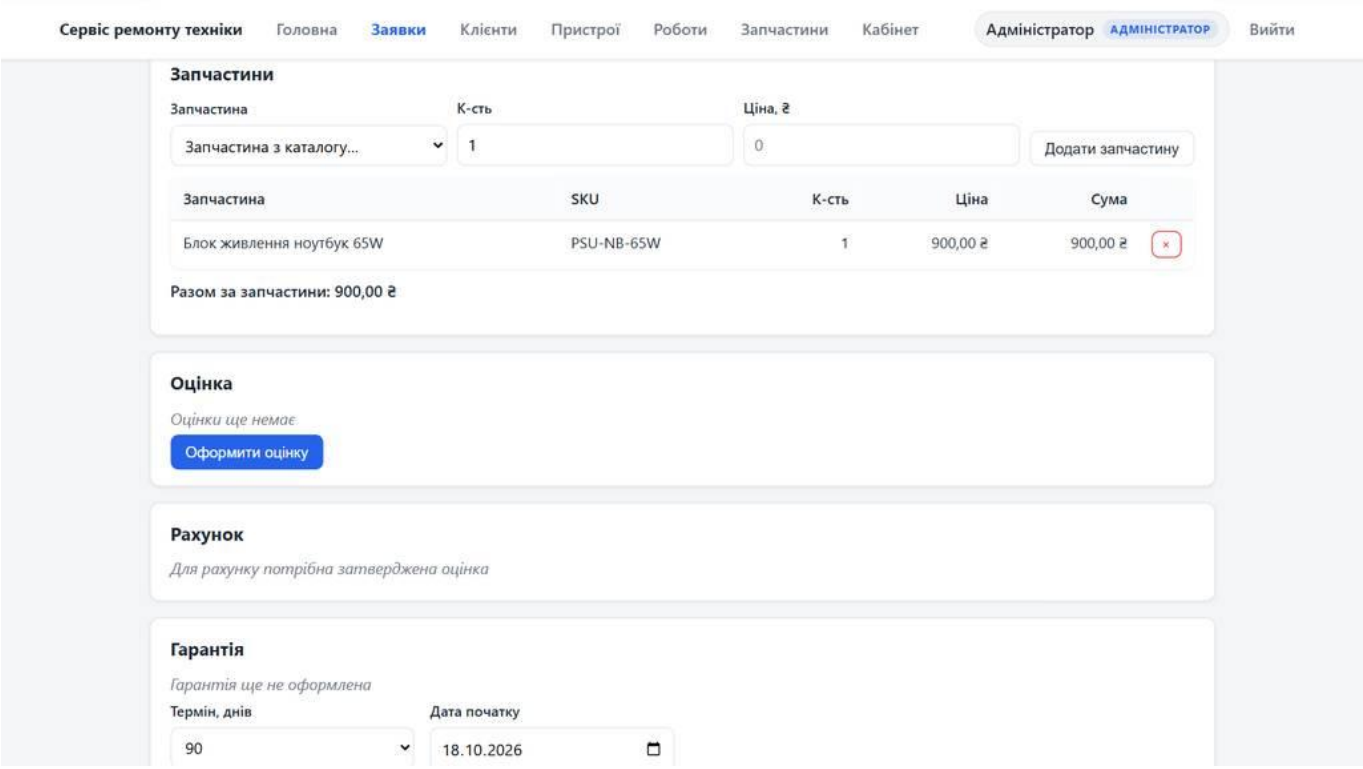


Права наростають за ролями: Відвідувач — лише перегляд; Майстер разом з Адміністратором — усі зміни (довідники, заявки, фінанси, технічні дії); Клієнт — особистий кабінет і власні замовлення. Усі мутуючі маршрути закривають одна й та сама матриця requireRole(ROLES.*). На скріншотах — список клієнтів та список заявок від імені персоналу.

Завдання 4: Оцінка: дія «Відхилити» видаляє чорновик оцінки з бази даних, заявка повертається в стан «Оцінки ще немає».

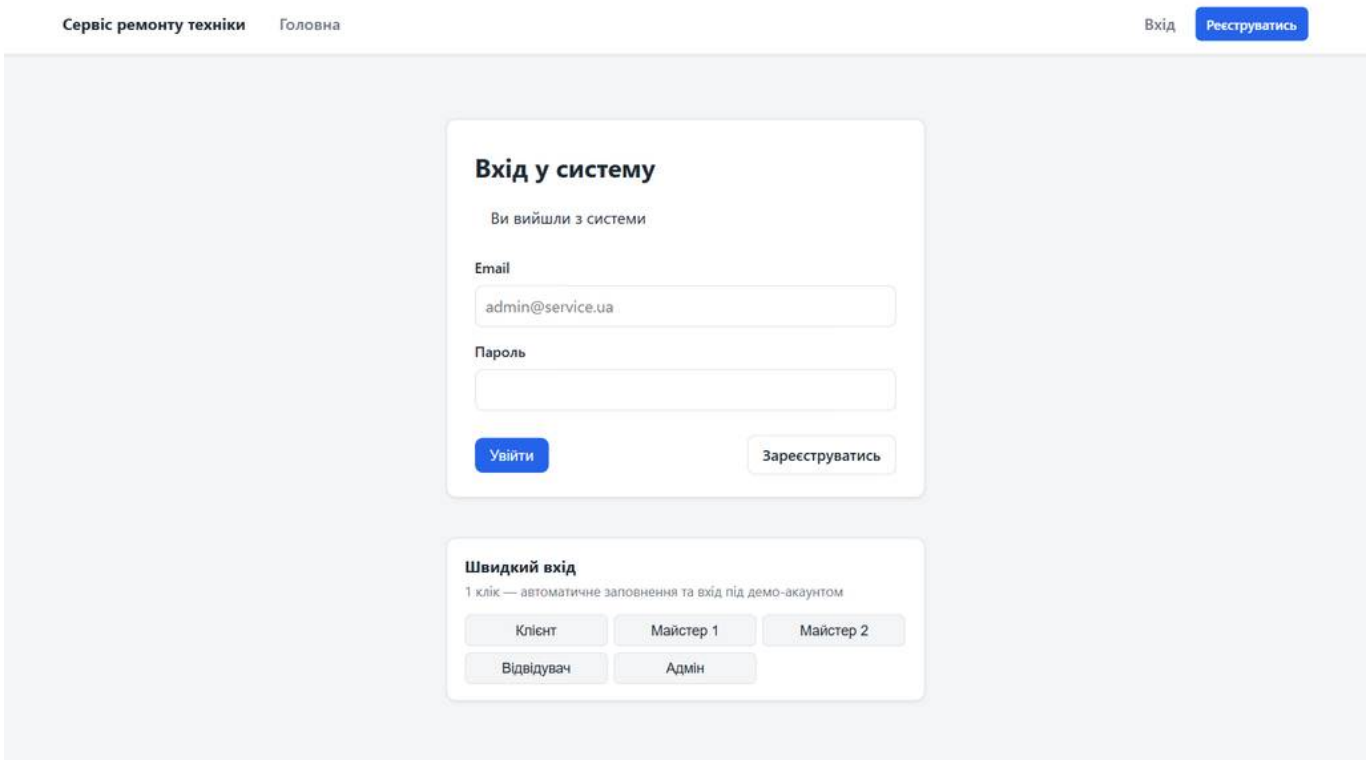
1. src/controllers/requests.ts — відхилення оцінки (DELETE)

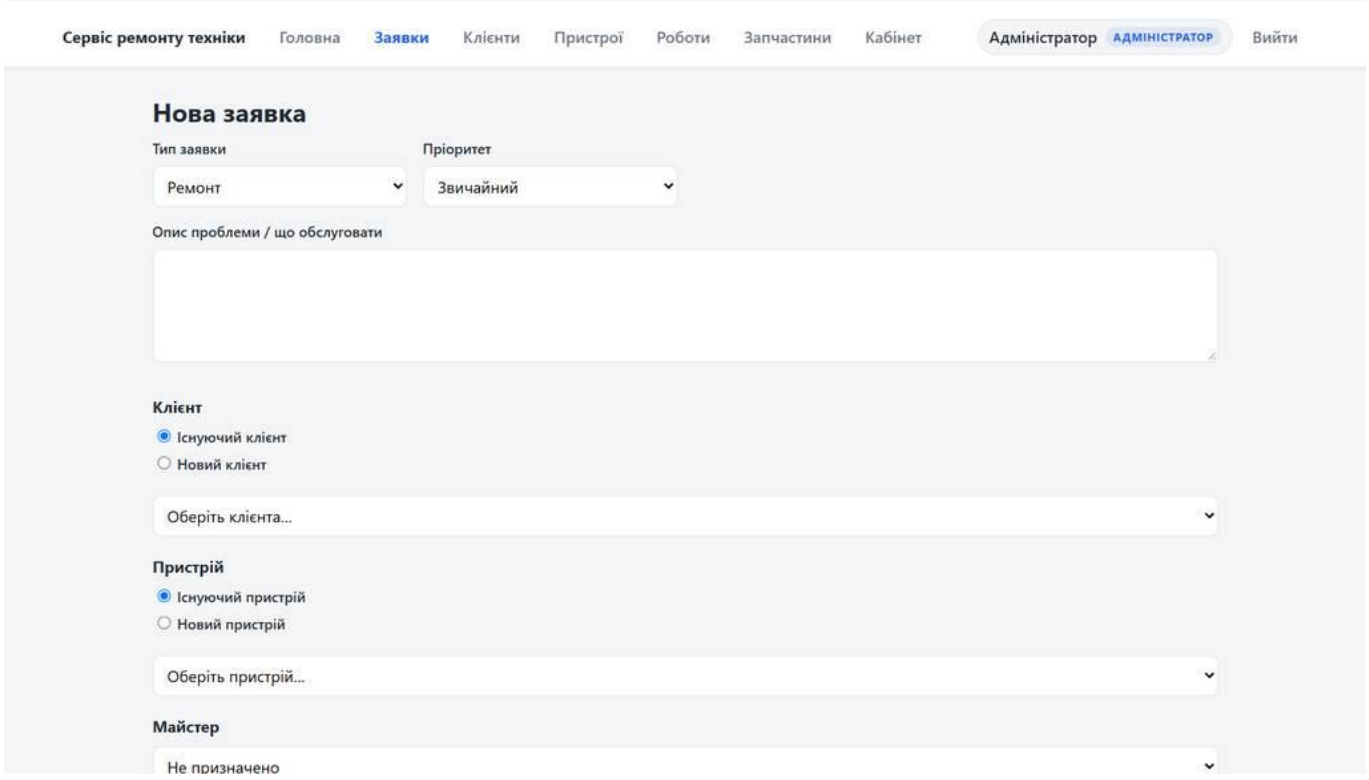
```
/** Відхилення оцінки: чорновик видаляється з БД, заявку можна оцінити заново. */
export async function rejectEstimate(req: Request, res: Response) {
  const id = req.params.id;
  const request = await getRequestWithRefs(id);
  if (!request || !request.estimate) {
    return res.status(404).render("errors/404", { title: "Заявку не знайдено", nav: "staff"
});
  }
  await prisma.estimate.delete({ where: { id: request.estimate.id } });
  return res.redirect(`/requests/${id}`);
}
```



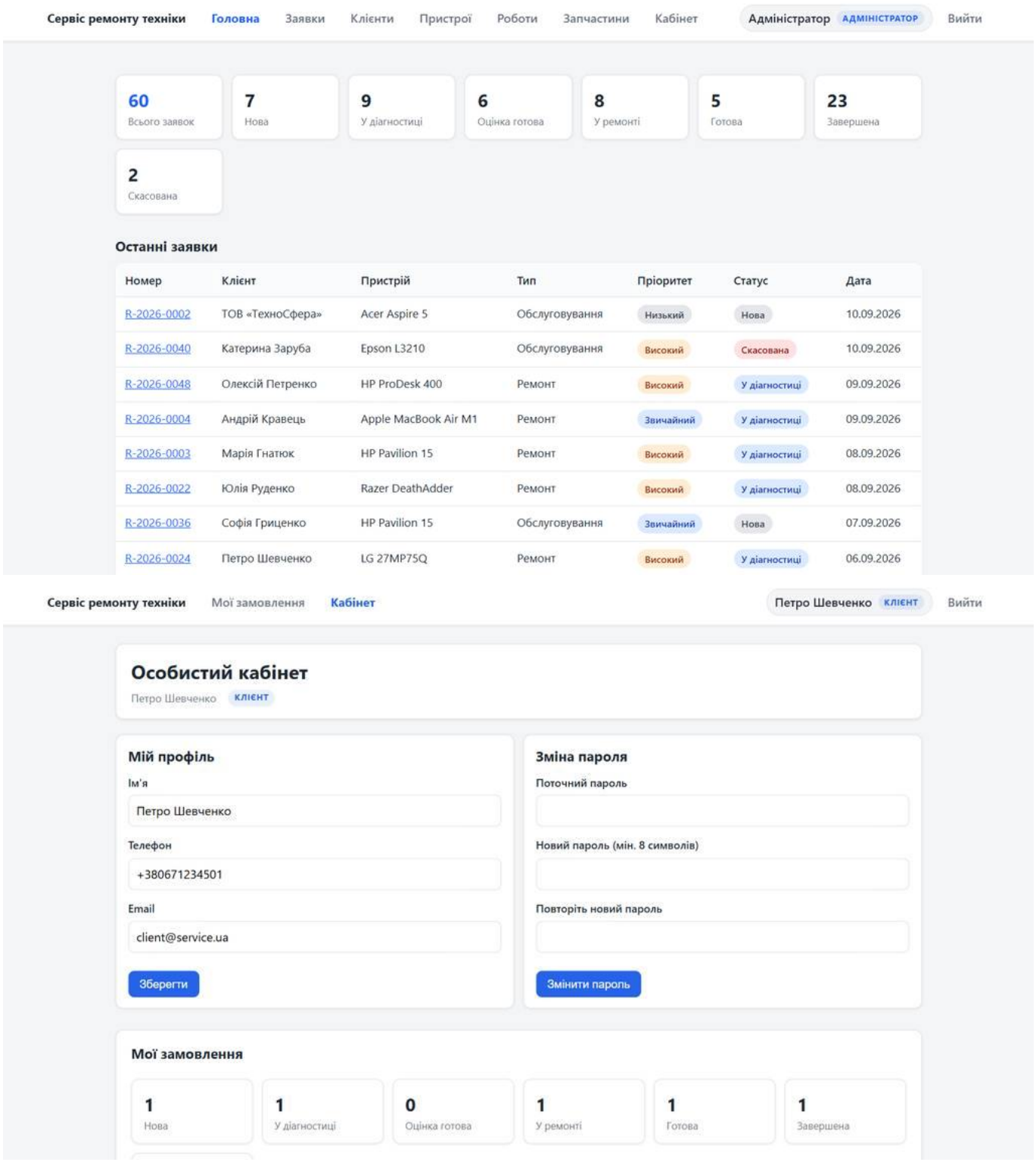
Після відхилення секція «Оцінка» на картці заявки показує «Оцінки ще немає» та кнопку «Оформити оцінку» — повторне оцінення цієї ж заявки можливе з чистого початку. Кнопка відхилення доступна лише персоналу (маршрут закриває requireRole(ROLES.STAFF)).

Завдання 5: Особистий кабінет — лише для ролі Клієнт (CLIENT); доопрацювання інтерфейсу основних сторінок.





Сторінка входу містить кнопки швидкого входу за демо-ролями, що полегшує перевірку прав доступу. Поля форм (наприклад, на сторінці створення заявки) займають повну ширину сторінки.



Дашборд починається одразу з блоків статистики. У кабінеті клієнта — картки профілю та зміни пароля і блок «Мої замовлення» зі статистикою власних заявок за статусами; гостю, персоналу та відвідувачу замість кабінету редіректує на домашню сторінку.

Висновок: На цій лабораторній роботі я реалізував авторизацію користувачів за допомогою Passport.js (локальна стратегія email+пароль, bcryptjs) та сесій express-session, міграцією додав четверту роль Відвідувач і заклав її в seed. Захист маршрутів зроблено middleware requireRole(roles): гостя редіректить на /login?next=..., а авторизованого користувача без потрібної ролі — на сторінку 403 із

зазначенням, яка роль потрібна. Усі чотири ролі перевірено на живому застосунку: відвідувач бачить лише перегляд, майстер разом із адміністратором керує заявками, довідниками та фінансами, клієнт — лише своїм кабінетом і власними замовленнями, адміністратор — має всі права. Закріпив механіку сесій (httpOnly-cookie, serialize/deserialize, роль з БД на кожному запиті) і паттерн «одна матриця дозволів на всіх мутуючих маршрутах».